SPECYFIKACKA DLA PROJEKTU

Zaawansowane techniki sztucznej inteligencji

1. Temat Projektu: FlowerCam – rozpoznawanie gatunków kwiatów w czasie rzeczywistym na iPhonie z wykorzystaniem transfer learningu (MobileNetV3, Core ML)

2. Realizacja samodzielna czy zespołowa

3. Osoba realizująca: Dawid Stefański, nr albumu: 122429

4. Pozostałe osoby zespołu: brak – projekt realizowany samodzielnie

5. Cel Projektu:

Celem projektu jest zbudowanie modelu głębokiego uczenia, który rozpoznaje gatunek kwiatu na obrazie, oraz wdrożenie go w aplikacji na iPhone'a działającej w czasie rzeczywistym – na żywo z kamery, w całości na urządzeniu, bez połączenia z internetem. Projekt pokazuje zastosowanie uczenia transferowego (transfer learning): sieć MobileNetV3 wstępnie wytrenowana na zbiorze ImageNet (1,28 mln zdjęć) jest dostrajana do nowego zadania przy niewielkim zbiorze danych (ok. 4,4 tys. zdjęć). Obejmuje pełną ścieżkę: przygotowanie danych, trening i porównanie modeli, ewaluację, konwersję do formatu Core ML oraz aplikację mobilną.

6. Założenia projektowe:

- Model rozpoznaje 5 gatunków kwiatów: stokrotkę, mniszek lekarski, różę, słonecznik i tulipan, oraz dodatkową klasę „to nie kwiat”.
- Dane kwiatów pochodzą z publicznego zbioru tf_flowers (3670 zdjęć). Zdjęcia klasy „to nie kwiat” pochodzą ze zbioru Caltech-101 (760 zdjęć z 95 kategorii, z wyłączeniem kategorii roślinnych).
- Dane są dzielone w sposób stratyfikowany na zbiór treningowy, walidacyjny i testowy w proporcji 70/15/15; zbiór testowy nie jest używany w trakcie treningu.

- Punktem wyjścia jest sieć MobileNetV3-Large z wagami ImageNet. Dla porównania trenowana jest od zera prosta sieć konwolucyjna (model bazowy).
- Trening odbywa się lokalnie na komputerze Mac z procesorem Apple Silicon (PyTorch z akceleracją GPU – MPS).
- Aplikacja docelowa działa na iPhonie z systemem iOS 17 lub nowszym; cała inferencja odbywa się na urządzeniu (Core ML, Neural Engine).
- Kwiat znajduje się w centralnej części kadru, zajmuje znaczną jego część, a scena jest dostatecznie oświetlona.
- Środowisko realizacji: Python 3.12, PyTorch 2.7, coremltools 9.0, Xcode, Swift i SwiftUI.

7. Wymagania:

Wymagania funkcjonalne

- **F1.** Aplikacja wyświetla podgląd na żywo z tylnej kamery telefonu.
- **F2.** Aplikacja klasyfikuje kolejne klatki obrazu i wyświetla nazwę rozpoznanego kwiatu (po polsku) wraz z poziomem pewności w procentach.
- **F3.** Aplikacja wyświetla trzy najbardziej prawdopodobne klasy z paskami prawdopodobieństwa.
- **F4.** Gdy w kadrze nie ma kwiatu, aplikacja wyświetla komunikat „To nie kwiat".
- **F5.** Gdy pewność predykcji jest niższa od progu, aplikacja wyświetla komunikat „Nie jestem pewien" zamiast nazwy kwiatu; próg jest regulowany suwakiem (40–95%, domyślnie 70%).
- **F6.** Aplikacja wyświetla bieżącą liczbę przetwarzanych klatek na sekundę (FPS) oraz czas inferencji jednej klatki (ms).
- **F7.** Aplikacja prosi o zgodę na dostęp do kamery i obsługuje odmowę (komunikat z przejściem do Ustawień).
- **F8.** Część uczenia maszynowego udostępnia skrypty przygotowania danych, treningu, ewaluacji i eksportu modelu do Core ML, uruchamiane jedną komendą.

Wymagania niefunkcjonalne

- **N1. Jakość:** dokładność (accuracy) modelu na zbiorze testowym ≥ 90% oraz macro F1 ≥ 0,90.
- **N2. Wydajność:** ≥ 15 klatek na sekundę na iPhonie, czas inferencji < 30 ms na klatkę.
- **N3. Rozmiar:** model w aplikacji ≤ 20 MB.
- **N4. Zgodność:** model Core ML zwraca tę samą klasę top-1 co model PyTorch dla ≥ 99% zdjęć testowych.
- **N5. Prywatność i praca offline:** obraz nie opuszcza urządzenia, aplikacja nie wymaga dostępu do internetu.

- **N6. Stabilność:** predykcje są wygładzane w czasie, aby wynik nie zmieniał się chaotycznie między klatkami.
- **N7. Powtarzalność:** stałe ziarno losowości, zapisany podział danych, przypięte wersje bibliotek.

Wymagania systemowe

- **S1. Trening:** komputer Mac z procesorem Apple Silicon (lub komputer z GPU NVIDIA), ok. 2 GB wolnego miejsca, Python 3.12 z bibliotekami PyTorch 2.7.0, torchvision 0.22.0, coremltools 9.0, scikit-learn, matplotlib.
- **S2. Budowanie aplikacji:** macOS z Xcode (iOS SDK 17 lub nowszy) oraz konto Apple ID do podpisania aplikacji.
- **S3. Urządzenie docelowe:** iPhone z systemem iOS 17 lub nowszym (procesor A12 Bionic lub nowszy, z Neural Engine) z tylną kamerą.

Wymagania użytkownika

- **U1.** Użytkownik rozpoznaje kwiat, kierując na niego aparat telefonu – bez robienia zdjęcia i bez dodatkowych kroków.
- **U2.** Wynik jest czytelny: nazwa kwiatu po polsku, procent pewności i kolor oznaczający stan (zielony – rozpoznano, żółty – niepewne, pomarańczowy – brak kwiatu).
- **U3.** Użytkownik wie, gdzie umieścić kwiat – na ekranie widoczna jest ramka celownika.
- **U4.** Aplikacja działa w terenie, bez dostępu do internetu.
- **U5.** Gdy model nie jest pewny, aplikacja informuje o tym zamiast podawać błędną nazwę.

8. Oczekiwany rezultat Projektu:

- Wytrenowany model klasyfikacji (MobileNetV3-Large po fine-tuningu) porównany z modelem bazowym (sieć CNN uczona od zera) i z wariantem z zamrożonym backbone'em.
- Raport z ewaluacji na zbiorze testowym: accuracy, macro F1, precision i recall dla każdej klasy, macierz pomyłek, krzywe uczenia oraz analiza progu pewności.
- Model w formacie Core ML (FlowerClassifier.mlpackage) zweryfikowany względem modelu PyTorch.
- Aplikacja iOS FlowerCam działająca na iPhonie, rozpoznająca kwiaty na żywo z kamery, w trybie offline.
- Kod źródłowy (Python i Swift) wraz z dokumentacją architektury i instrukcją uruchomienia.

- **Wyniki uzyskane na zbiorze testowym (665 zdjęć):** accuracy 95,5% i macro F1 0,953 dla MobileNetV3 po fine-tuningu, wobec 82,9% dla modelu bazowego; model Core ML zgodny z PyTorch dla 665/665 zdjęć, rozmiar 8,1 MB.

9. Kamień milowy Projektu:

Projekt podzielono na cztery kamienie milowe; każdy ma mierzalne kryterium zaliczenia:

Kamień milowy	Metryki (kryterium)	Sposób walidacji	Wynik
KM1. Przygotowanie danych i model bazowy	Podział 70/15/15 (3101/664/665 zdjęć); accuracy modelu bazowego na zbiorze testowym	Stratyfikowany, niezależny zbiór testowy; zapisany podział danych (splits.json)	82,9%
KM2. Model z transfer learningiem	Accuracy ≥ 90% i macro F1 ≥ 0,90 na zbiorze testowym	Skrypt ewaluacji: accuracy, macro F1, precision/recall dla klas, macierz pomyłek; porównanie z KM1	95,5% / 0,953
KM3. Eksport modelu do Core ML	Zgodność klasy top-1 z modelem PyTorch ≥ 99%; rozmiar modelu ≤ 20 MB	Predykcja modelem PyTorch i Core ML na całym zbiorze testowym i porównanie wyników	100% (665/665), 8,1 MB
KM4. Aplikacja iOS w czasie rzeczywistym	≥ 15 FPS; czas inferencji < 30 ms na klatkę; poprawne rozpoznawanie kwiatów; działanie offline	Liczniki FPS i ms w aplikacji; test na zdjęciach kwiatów ze zbioru testowego pokazanych kamerze; test w trybie samolotowym	ok. 30 FPS (iPhone 17)